

Quadray Analytical Details and Methods

Daniel Ari Friedman
ORCID: 0000-0001-6232-9096
Email: daniel@activeinference.institute
DOI: 10.5281/zenodo.16887791

October 07, 2026

Contents

1 Quadray Analytical Details and Methods	1
1.1 Overview	1
1.2 Mathematical Foundations	1
1.2.1 Framework Distinctions	1
1.2.2 Key Mathematical Principles	1
1.2.3 Implementation Strategy	1
1.3 Framework Integration	2
1.4 Fuller.4D Coordinates and Normalization	2
1.5 Conversions and Vector Operations: Quadray $\leftrightarrow$ Cartesian (Fuller.4D $\leftrightarrow$ Coxeter.4D/XYZ)	2
1.5.1 Integer-coordinate constructions (compact derivation box)	2
1.5.2 Example vertex lists and volume checks (illustrative)	4
1.6 Integer Volume Quantization	4
1.7 Distances and Metrics	5
1.8 XYZ determinant and S3 conversion	5
1.8.1 Alternative Volume Formulas	6
1.9 Fisher Geometry in Quadray Space	6
1.10 Practical Methods	6
1.11 Tetravolumes with Quadrays	6
1.11.1 Bridging and native tetravolume formulas	7
1.11.2 Short Python snippets	7
1.11.3 Random tetrahedra in the IVM (integer volumes)	10
1.11.4 Algebraic precision	11
1.11.5 XYZ determinant and the S3 conversion	11
1.11.6 D^3 vs R^3 : 60° “closing the lid” vs orthogonal “cubing”	11
1.12 Practical Implementation Workflow	11
1.12.11. Mathematical Formulation	11
1.12.22. Implementation Strategy	11
1.12.33. Testing and Validation	12
1.12.44. Documentation and Reproducibility	12
1.12.55. Optimization and Extension	12
1.13 Code methods (anchors)	12
1.13.1 Core Quadray operations	12
1.13.2 Quaternion operations	12
1.13.3 Quaternion metrics	14
1.13.4 Volume calculations	14
1.13.5 Coordinate conversions	14
1.13.6 Linear algebra utilities	14
1.13.7 Optimization methods	14

1.13.8	Information geometry methods	15
1.13.9	Metrics and analysis	15
1.13.10	Examples and utilities	16
1.13.11	Symbolic computation	16
1.13.12	Visualization and animation	16
1.13.13	Path and file utilities	16
1.13.14	Additional Nelder-Mead components	17
1.13.15	Discrete variational components	17
1.13.16	Glossary generation	17
1.13.17	Geometry utilities	17
1.13.18	Comprehensive test coverage	17
1.14	Reproducibility checklist	18

1 Quadray Analytical Details and Methods

1.1 Overview

This section provides detailed analytical methods for working with Quadray coordinates, including coordinate conventions, volume calculations, and optimization approaches. We emphasize the distinction between different 4D frameworks and provide practical computational methods.

1.2 Mathematical Foundations

The methods presented here rest on three interconnected mathematical frameworks. For comprehensive definitions, see [Section 2: 4D Namespaces](#).

1.2.1 Framework Distinctions

- **Coxeter.4D**: Euclidean 4D geometry with standard metric tensor and volume forms
- **Einstein.4D**: Minkowski spacetime with indefinite metric for information geometry analogies
- **Fuller.4D**: Synergetics/Quadray coordinates with integer lattice constraints and IVM unit conventions

1.2.2 Key Mathematical Principles

- **Rational volume quantization**: Lattice constraints quantize tetrahedral volumes to exact quarter-unit ($\frac{1}{4}$ -grain) rationals in IVM units — integral for unit-tetra tilings, e.g. $\frac{1}{4}$ for the primitive tetrahedron
- **Coordinate system bridges**: Linear transformations between Fuller.4D and Coxeter.4D preserve geometric relationships
- **Information geometry**: Fisher metric provides Riemannian structure for optimization on parameter manifolds
- **Exact arithmetic**: Bareiss algorithm ensures determinant calculations remain exact for integer inputs

1.2.3 Implementation Strategy

All mathematical concepts are implemented with: - **Exact arithmetic** where possible (integer determinants, symbolic computation) - **Numerical stability** for floating-point operations (ridge regularization, condition number monitoring) - **Cross-validation** between different formulations (Ace 5×5 vs Cayley-Menger, native vs bridging approaches) - **Comprehensive testing** ensuring mathematical correctness across edge cases

1.3 Framework Integration

The three 4D frameworks serve distinct but complementary roles in our implementation:

- **Coxeter.4D provides the container:** Euclidean 4D space serves as the mathematical foundation for volume calculations, distance metrics, and geometric transformations. When we compute volumes via Cayley-Menger determinants or XYZ coordinate determinants, we operate in this framework.
- **Einstein.4D provides the analogy:** The Minkowski metric structure inspires our information geometry approach, where the Fisher information matrix acts as a Riemannian metric on parameter space. Natural gradient descent follows geodesics on this information manifold, analogous to how particles follow geodesics in spacetime.
- **Fuller.4D provides the constraints:** The Quadray coordinate system and IVM lattice impose integer constraints that enable exact arithmetic and discrete optimization. The synergetics unit conventions (regular tetrahedron volume = 1) create a quantized geometry where volumes are exact rationals on the $1/4$ grid — integral for tetrahedra tiling unit IVM tetras, $\frac{1}{4}$ -grain otherwise.

This multi-framework approach allows us to: 1. Use standard Euclidean methods for volume calculations where relevant or already in use (Coxeter.4D) 2. Apply information geometry principles for geodesic optimization (Einstein.4D analogy) 3. Maintain exact arithmetic in the Isotropic Vector Matrix (IVM) setting through integer lattice constraints (Fuller.4D) 4. Bridge and swap among frameworks via coordinate transformations and unit conversions

1.4 Fuller.4D Coordinates and Normalization

- Quadray vector $q = (a,b,c,d)$, $a,b,c,d \geq 0$, with at least one coordinate zero under normalization.
- Projective normalization can add/subtract (k,k,k,k) without changing direction; choose k to enforce non-negativity and one zero minimum.
- Isotropic Vector Matrix (IVM): integer quadrays describe CCP sphere centers; the 12 permutations of $\{2,1,1,0\}$ form the cuboctahedron (vector equilibrium).
 - Integer-coordinate models: assigning unit IVM tetravolume to the regular tetrahedron yields integer coordinates for several familiar polyhedra (inverse tetrahedron, cube, octahedron, rhombic dodecahedron, cuboctahedron) when expressed as linear combinations of the four quadray basis vectors. See overview: [Quadray coordinates](#).

1.5 Conversions and Vector Operations: Quadray $\leftrightarrow$ Cartesian (Fuller.4D $\leftrightarrow$ Coxeter.4D/XYZ)

- **Embedding conventions** determine the linear maps between Quadray (Fuller.4D) and Cartesian XYZ (a 3D slice or embedding aligned with Coxeter.4D conventions).
- **References:** Urner provides practical conversion write-ups and matrices; see:
 - Quadrays and XYZ: [Urner - Quadrays and XYZ](#)
 - Introduction with examples: [Urner - Quadray intro](#)
- **Implementation:** choose a fixed tetrahedral embedding; construct a 3×4 matrix M that maps (a,b,c,d) to (x,y,z) , respecting A,B,C,D directions to tetra vertices. The inverse map can be defined up to projective normalization (adding (k,k,k,k)). When comparing volumes, use the $S_3 = \sqrt{9/8}$ scale to convert XYZ (Euclidean) volumes to IVM (Fuller.4D) units.
- **Vector view:** treat q as a vector with magnitude and direction; define dot products and norms by pushing to XYZ via M .

1.5.1 Integer-coordinate constructions (compact derivation box)

- Under the synergetics convention (unit regular tetrahedron has tetravolume 1), many familiar solids admit Quadray integer coordinates. For example, the octahedron at the same edge length has tetravolume 4, and its vertices can be formed as integer linear combinations of the four axes A,B,C,D subject to the Quadray normalization rule.
- The cuboctahedron (vector equilibrium) arises as the shell of the 12 nearest IVM neighbors given by the permutations of $(2, 1, 1, 0)$. The rhombic dodecahedron (tetravolume 6) is the Voronoi cell of the FCC/CCP packing centered at the origin under the same embedding.

- See the following figure for a schematic summary of these relationships.

Object	Quadray construction (sketch)	IVM volume
Regular tetrahedron	Vertices $o=(0,0,0,0)$, $p=(2,1,0,1)$, $q=(2,1,1,0)$, $r=(2,0,1,1)$	1
Cube (same edge)	Union of 3 mutually orthogonal rhombic belts wrapped on the tetra frame; edges tracked by XYZ embedding; compare the following figure	3
Octahedron (same edge)	Convex hull of mid-edges of the tetra frame (pairwise axis sums normalized)	4
Rhombic dodecahedron	Voronoi cell of FCC/CCP packing at origin (dual to cuboctahedron)	6
Cuboctahedron (vector equilibrium)	Shell of the 12 nearest IVM neighbors: permutations of $(2,1,1,0)$	20
Truncated octahedron	Archimedean solid with 6 square and 8 hexagonal faces; space-filling tiling	20

Small coordinate examples (subset):

- Cuboctahedron neighbors (representatives): $(2,1,1,0)$, $(2,1,0,1)$, $(2,0,1,1)$, $(1,2,1,0)$; the full shell is all distinct permutations.
- Tetrahedron: $[(0,0,0,0)$, $(2,1,0,1)$, $(2,1,1,0)$, $(2,0,1,1)]$.

Short scripts:

```
1 python3 quadmath/scripts/polyhedra_quadray_constructions.py
```

Programmatic check (neighbors, equal radii, adjacency):

```
1 import numpy as np
2 from quadmath.core.examples import example_cuboctahedron_vertices_xyz
3
4 xyz = np.array(example_cuboctahedron_vertices_xyz())
5 r = np.linalg.norm(xyz[0])
6 assert np.allclose(np.linalg.norm(xyz, axis=1), r)
7
8 # Touching neighbors have separation 2r
9 touch = []
10 for i in range(len(xyz)):
11     for j in range(i+1, len(xyz)):
12         d = np.linalg.norm(xyz[i] - xyz[j])
13         if abs(d - 2*r) / (2*r) < 0.05:
14             touch.append((i, j))
15 assert len(touch) > 0
```

1.5.2 Example vertex lists and volume checks (illustrative)

The following snippets show executable decompositions for standard synergetics volumes. Each tetra volume is computed via `ace_tetravolume_5x5` and summed.

Octahedron ($V = 4$): the octahedron with vertices $\pm 2\hat{e}_i$ (edge $2\sqrt{2}$, matching the unit IVM tetra) decomposes into eight origin-apex orthant tetras, each of volume $\frac{1}{2}$:

```

1 from quadmath.core.quadray import Quadray, ace_tetravolume_5x5
2
3 o = Quadray(0,0,0,0)
4 axes = [
5     Quadray(1,0,0,1), Quadray(0,1,1,0),    # +x, -x
6     Quadray(1,1,0,0), Quadray(0,0,1,1),    # +y, -y
7     Quadray(1,0,1,0), Quadray(0,1,0,1),    # +z, -z
8 ]
9 V_oct = sum(
10     ace_tetravolume_5x5(o, x, y, z)
11     for x in axes[0:2] for y in axes[2:4] for z in axes[4:6]
12 ) # 8 * 1/2 = 4

```

Cube ($V = 3$): the cube with XYZ vertices $(\pm 1, \pm 1, \pm 1)$ (edge 2) decomposes into the inscribed tetra on four alternating cube vertices ($V = 1$) plus four corner tetras ($V = \frac{1}{2}$ each):

```

1 from quadmath.core.quadray import Quadray, ace_tetravolume_5x5
2
3 inner = ace_tetravolume_5x5(
4     Quadray(1,0,0,0), Quadray(0,1,0,0), Quadray(0,0,1,0), Quadray(0,0,0,1),
5 ) # 1
6 corner = ace_tetravolume_5x5(
7     Quadray(0,1,1,1), Quadray(0,0,0,1), Quadray(0,1,0,0), Quadray(0,0,1,0),
8 ) # 1/2
9 V_cube = inner + 4 * corner # 1 + 4*(1/2) = 3

```

Notes.

- The octahedron and cube decompositions above execute exactly as shown (verified against `ace_tetravolume_5x5`). Other equivalent tilings are possible.
- Volumes are invariant to adding (k, k, k, k) to each vertex of a tetra (projective normalization), which the 5×5 determinant respects.

1.6 Integer Volume Quantization

For a tetrahedron with vertices $P_0..P_3$ in the Quadray integer lattice (Fuller.4D), see Eq. (??) in the equations appendix.

- With integer quadray coordinates the projected determinant is an exact integer, and the IVM tetravolume is the exact rational $|\det|/4$ (see Eq. (??)), implemented with `Fractions.Fraction` in `integer_tetra_volume` and `ace_tetravolume_5x5`. Volumes are integral for tetrahedra that tile into unit IVM tetras (determinant divisible by 4) and fractional otherwise: the primitive tetrahedron spanned by $(0, 0, 0, 0)$, $(1, 0, 0, 0)$, $(0, 1, 0, 0)$, $(0, 0, 1, 0)$ has determinant 1 and exact volume $\frac{1}{4}$.
- Unit conventions: the unit IVM tetrahedron (origin plus three IVM neighbor moves, permutations of $(2, 1, 1, 0)$) has determinant 4 and volume exactly 1 (synergetics).

Notes.

- $P_0, \dots, P_3$ are tetrahedron vertices in Quadray coordinates.
- V_{xyz} in Eq. (??) is the Euclidean (XYZ) volume; converting it to IVM tetra-units requires the synergetics scale factor $S3 = \sqrt{9/8}$ in sphere-radius units (Eq. (??)). For lattice tetrahedra, the direct quadray formula Eq. (??) yields the IVM volume exactly, without unit conversion.
- Background and variations are discussed under Tetrahedron volume formulas: [Tetrahedron - volume](#).

Tom Ace 5×5 determinant (tetravolume directly from quadrays), see Eq. (??) in the equations appendix.

This returns the same exact tetravolumes as `integer_tetra_volume` for every lattice tetrahedron: $|\det_{5 \times 5}|$ always equals the magnitude of the projected 3×3 determinant, so both implementations agree exactly (both return `fractions.Fraction` values $|\det|/4$).

Notes.

- Rows correspond to the Quadray 4-tuples of the four vertices with a final affine column of ones; the last row enforces projective normalization.
- The factor $\frac{1}{4}$ returns tetravolumes in IVM units consistent with synergetics. See also [Quadray coordinates](#).

Equivalently, define the 5×5 matrix of quadray coordinates augmented with an affine 1 as shown in Eq. (??) in the equations appendix.

Points vs vectors: subtracting points is shorthand for forming edge vectors. We treat quadray 4-tuples as vectors from the origin; differences like $(P_1 - P_0)$ mean “edge vectors,” avoiding ambiguity between “points” and “vectors.”

Equivalently via Cayley-Menger determinant (Coxeter.4D/Euclidean lengths) ([Cayley-Menger determinant](#)), see Eq. (??) in the equations appendix.

References: [Cayley-Menger determinant](#), lattice tetrahedra discussions in geometry texts; see also [Tetrahedron - volume](#). Code: `integer_tetra_volume`, `ace_tetravolume_5x5`.

Notes.

- **Pairwise distances:** d_{ij} are Euclidean distances between vertices P_i and P_j .
- **Length-only formulation:** Cayley-Menger provides a length-only formula for simplex volumes, here specialized to tetrahedra; see the canonical reference above.

Table 1: Polyhedra tetravolumes in IVM units (edge length equal to the unit tetra edge).

Polyhedron (edge = tetra edge)	Volume (tetra-units)
Regular Tetrahedron	1
Cube	3
Octahedron	4
Rhombic Dodecahedron	6
Cuboctahedron (Vector Equilibrium)	20
Truncated Octahedron	20

1.7 Distances and Metrics

Distance definitions depend on the chosen embedding and normalization. For cross-references to information geometry, see [Eq. \(FIM\)](#) and [natural gradient](#) in the Equations appendix.

1.8 XYZ determinant and S3 conversion

Given XYZ coordinates of tetrahedron vertices (x_i, y_i, z_i), the Euclidean volume is computed as shown in Eq. (??) in the equations appendix.

Convert to IVM units via $V_{ivm} = S3 \cdot V_{xyz}$ with $S3 = \sqrt{9/8}$. See background discussion under [Tetrahedron - volume](#).

1.8.1 Alternative Volume Formulas

- **Piero della Francesca (PdF) formula**, which consumes edge lengths and returns Euclidean volumes. See Eq. (??) in the equations appendix.

Convert to IVM units via $V_{ivm} = S3 \cdot V_{xyz}$ with $S3 = \sqrt{9/8}$. See background discussion under [Tetrahedron - volume](#).

- **Gerald de Jong (GdJ) formula**, which natively returns tetravolumes. See Eq. (??) in the equations appendix.

In Quadray coordinates, one convenient native form uses edge-vector differences and an integer-preserving determinant (agreeing with Ace 5×5):

where each column is formed from Quadray component differences of $P_1 - P_0, P_2 - P_0, P_3 - P_0$ projected via $\pi(P) = (a - d, b - d, c - d)$ to the synergetics 3D slice (matching `integer_tetra_volume`); integer arithmetic is exact and the factor $\frac{1}{4}$ produces IVM tetravolumes. See de Jong's Quadray notes and Urner's implementations for derivations ([Quadray coordinates](#)).

- Euclidean embedding distance via appropriate linear map from quadray to R^3 .
- Information geometry metric: Fisher Information Matrix (FIM)
 - $FIM[i, j] = \mathbb{E}[\partial_{\theta_i} \log p(x; \theta) \partial_{\theta_j} \log p(x; \theta)]$
 - Acts as Riemannian metric; natural gradient uses $FIM^{-1} \nabla \theta$. See [Fisher information](#).

1.9 Fisher Geometry in Quadray Space

- Symmetries of quadray lattices often induce near block-diagonal FIM.
- Determinant and spectrum characterize conditioning and information concentration.

1.10 Practical Methods

1.11 Tetravolumes with Quadrays

- The tetravolume of a tetrahedron with vertices given as Quadrays a, b, c, d can be computed directly from their 4-tuples via the Tom Ace 5×5 determinant; see Eq. (??) for the canonical form.
- Unit regular tetrahedron from origin: with $o=(0,0,0,0)$, $p=(2,1,0,1)$, $q=(2,1,1,0)$, $r=(2,0,1,1)$, we have $V_{ivm}(o,p,q,r)=1$. Doubling each vector scales volume by 8, as expected.
- Equivalent length-based formulas agree with the 5×5 determinant:

- Cayley-Menger: $288 V^2 = \det \begin{pmatrix} 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & d_{01}^2 & d_{02}^2 & d_{03}^2 \\ 1 & d_{10}^2 & 0 & d_{12}^2 & d_{13}^2 \\ 1 & d_{20}^2 & d_{21}^2 & 0 & d_{23}^2 \\ 1 & d_{30}^2 & d_{31}^2 & d_{32}^2 & 0 \end{pmatrix}.$

- Piero della Francesca (PdF) Heron-like formula (converted to IVM via $S3 = \sqrt{9/8}$).

Let edge lengths meeting at a vertex be a, b, c , and the opposite edges be d, e, f . The Euclidean volume satisfies

Convert to IVM units via $V_{ivm} = S3 \cdot V_{xyz}$ with $S3 = \sqrt{9/8}$. See background discussion under [Tetrahedron - volume](#).

- Gerald de Jong (GdJ) formula, which natively returns tetravolumes.

In Quadray coordinates, one convenient native form uses edge-vector differences and an integer-preserving determinant (agreeing with Ace 5×5):

where each column is formed from Quadray component differences of $P_1 - P_0, P_2 - P_0, P_3 - P_0$ projected via $\pi(P) = (a - d, b - d, c - d)$ to the synergetics 3D slice (matching `integer_tetra_volume`); integer arithmetic is exact and the factor $\frac{1}{4}$ produces IVM tetravolumes. See de Jong's Quadray notes and Urner's implementations for derivations ([Quadray coordinates](#)).

1.11.1 Bridging and native tetravolume formulas

- **Lengths (bridging):** PdF and Cayley-Menger (CM) consume Cartesian lengths (XYZ) and produce Euclidean volumes; convert to IVM units via $S3 = \sqrt{9/8}$.
- **Quadray-native:** Gerald de Jong (GdJ) returns IVM tetravolumes directly (no XYZ bridge). Tom Ace's 5×5 coordinate formula is likewise native IVM. All agree numerically with CM+S3 on shared cases.

References and discussion: [Uerner - Flickr diagram](#). For computational implementations and educational materials, see the [Resources](#) section.

Figure: automated comparison (native Ace 5×5 vs CM+S3) across small examples (see script `sympy_formalisms.py`). The figure and source CSV/NPZ are in `quadmath/output/`.

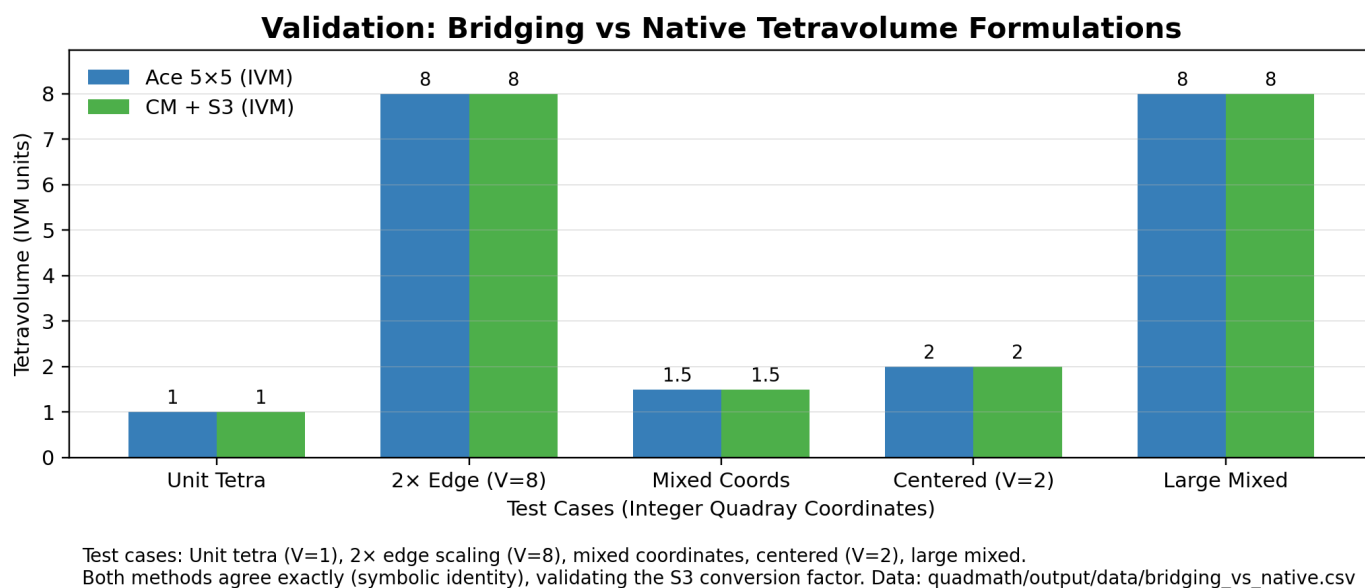


Figure 1: **Validation of bridging vs native tetravolume formulations across canonical examples.** This bar chart compares IVM tetravolumes computed via two independent methods: the “bridging” approach using Cayley-Menger determinants on Euclidean edge lengths converted to IVM units via the synergetics factor $S3 = \sqrt{9/8}$, versus the “native” approach using Tom Ace’s 5×5 determinant formula that operates directly on Quadray coordinates without XYZ intermediates. **Test cases:** Unit tetrahedron (V=1), 2× edge scaling (V=8), mixed coordinate tetrahedron, centered tetrahedron (V=3), and large mixed tetrahedron, all using integer Quadray coordinates. **Results:** The overlapping bars demonstrate numerical agreement at machine precision between the length-based Coxeter.4D approach (Cayley-Menger + S3 conversion) and the coordinate-based Fuller.4D approach (Ace 5×5), confirming the mathematical equivalence of these formulations under synergetics unit conventions. Raw numerical data saved as `bridging_vs_native.csv` for reproducibility and further analysis.

1.11.2 Short Python snippets

```
1 from quadmath.core.quadray import Quadray, ace_tetravolume_5x5
2
3 o = Quadray(0,0,0,0)
4 p = Quadray(2,1,0,1)
5 q = Quadray(2,1,1,0)
6 r = Quadray(2,0,1,1)
```

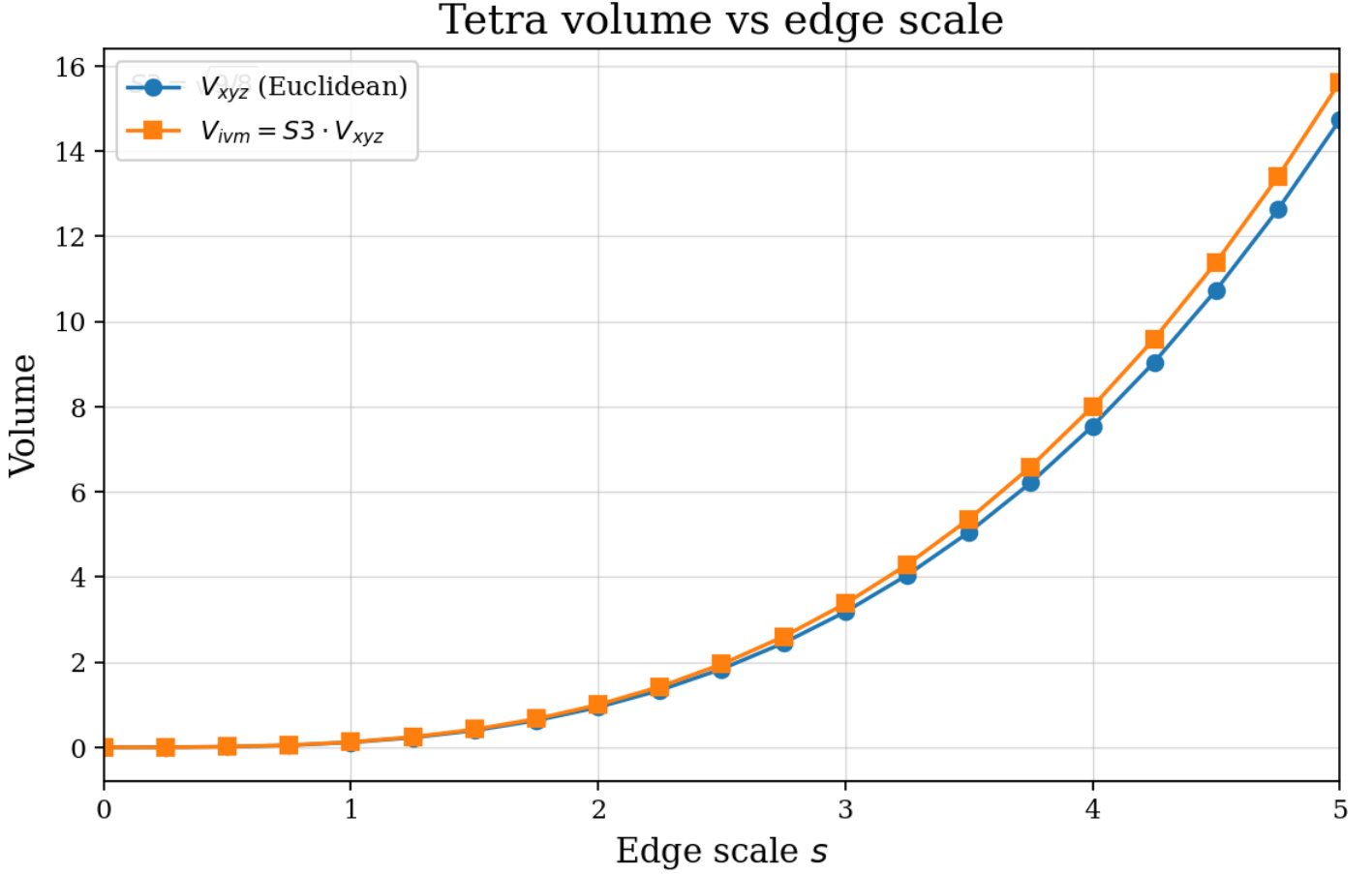
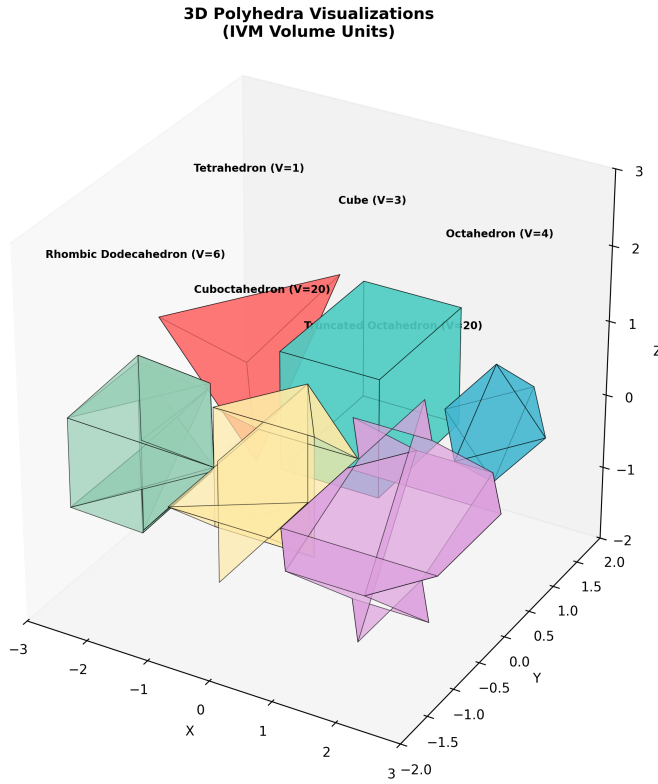
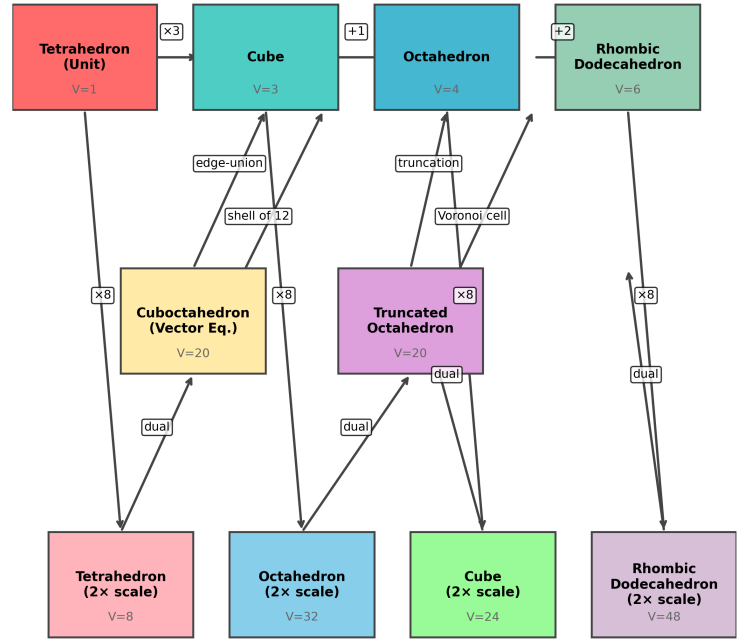


Figure 2: **Tetrahedron volume scaling relationships: Euclidean vs IVM unit conventions.** This plot demonstrates the mathematical relationship between edge length scaling and tetrahedron volume under both Euclidean (XYZ) and IVM (synergetics) unit conventions. **X-axis:** Edge length scaling factor (0 to 5.0). **Y-axis:** Tetrahedron volume in respective units. **Blue line (Euclidean):** Volume scales as the cube of edge length, following the standard $V = \frac{\sqrt{2}}{12} \cdot L^3$ relationship for regular tetrahedra. **Orange line (IVM):** Volume scales as the cube of edge length but in IVM tetra-units, following $V_{ivm} = \frac{1}{8} \cdot L^3$ where the regular tetrahedron with unit edge has volume 1/8. **Key insight:** The ratio between these two scaling laws is the synergetics factor $S3 = \sqrt{9/8} \approx 1.06066$, which converts between Euclidean and IVM volume conventions. **Mathematical foundation:** This scaling relationship demonstrates how both conventions preserve the cubic scaling relationship, but with different fundamental units reflecting the different geometric assumptions of Coxeter.4D (Euclidean) versus Fuller.4D (synergetics) frameworks. The plot provides the theoretical foundation for understanding volume conversions and scaling behavior in the IVM system.



**Synergetic Polyhedra Volume Relationships
Quadray/IVM Framework Network Diagram**



Volume relationships in IVM tetra-units. Regular tetrahedron ($V=1$) is the fundamental unit.
Scaling relationships: $2 \times \text{edge length} \rightarrow 8 \times \text{volume}$ ($V \propto L^3$).
Geometric constructions: edge-union, truncation, dual relationships, and Voronoi cells.
All polyhedra constructed with consistent edge lengths and integer Quadray coordinates.

Figure 3: Synergetic polyhedra volume relationships in the Quadray/IVM framework (comprehensive visualization). This figure combines 3D polyhedra visualizations with an extended network diagram showing integer volume relationships among key synergetic polyhedra. **Left panel (3D visualizations):** Color-coded polyhedra including regular tetrahedron ($V=1$, fundamental unit), cube ($V=3$), octahedron ($V=4$), rhombic dodecahedron ($V=6$), cuboctahedron ($V=20$), and truncated octahedron ($V=20$), all constructed with consistent edge lengths and proper geometric faces. **Right panel (network diagram):** Extended volume relationships showing fundamental shapes ($V=1, 3, 4, 6$), complex constructions ($V=20$), and scaling relationships ($2 \times \text{edge length} \rightarrow 8 \times \text{volume}$). **Additional polyhedra:** Includes truncated octahedron ($V=20$) and scaled variants demonstrating the “third power” volume scaling law $V \propto L^3$ in IVM units. **Geometric constructions:** Edge-union relationships, truncation operations, dual polyhedra, and Voronoi cell constructions. **Fuller.4D significance:** These integer volume ratios reflect the quantized nature of space-filling in synergetics, where the regular tetrahedron provides a natural unit container and other polyhedra emerge as integer multiples, supporting discrete geometric computation and exact lattice-based optimization methods. All constructions respect the IVM unit convention where the regular tetrahedron has tetravolume 1.

```
7 assert ace_tetrvolume_5x5(o,p,q,r) == 1 # unit IVM tetra
```

```
1 import numpy as np
2 from quadmath.core.cayley_menger import ivm_tetra_volume_cayley_menger
3
4 # Example: regular tetrahedron with edge length 1 (XYZ units)
5 d2 = np.ones((4,4)) - np.eye(4) # squared distances
6 V_ivm = ivm_tetra_volume_cayley_menger(d2) # = 1/8 in IVM tetra-units
```

```
1 # SymPy implementation of Tom Ace 5x5 (symbolic determinant)
2 from sympy import Matrix
3
4 def qvolume(q0, q1, q2, q3):
5     M = Matrix([
6         q0 + (1,),
7         q1 + (1,),
8         q2 + (1,),
9         q3 + (1,),
10        [1, 1, 1, 1, 0],
11    ])
12    return abs(M.det()) / 4
```

```
1 # Symbolic variant with SymPy (exact radicals)
2 from sympy import Matrix, sqrt, simplify
3 from quadmath.core.symbolic import cayley_menger_volume_symbolic, convert_xyz_volume_to_ivm_symbolic
4
5 d2 = Matrix([[0,1,1,1],[1,0,1,1],[1,1,0,1],[1,1,1,0]])
6 V_xyz_sym = cayley_menger_volume_symbolic(d2) # sqrt(2)/12
7 V_ivm_sym = simplify(convert_xyz_volume_to_ivm_symbolic(V_xyz_sym)) # 1/8
```

1.11.3 Random tetrahedra in the IVM (integer volumes)

- The 12 CCP directions are the permutations of (2,1,1,0). Random walks on this move set generate integer-coordinate Quadrays; resulting tetrahedra have integer tetravolumes.

```
1 from itertools import permutations
2 from random import choice
3 from quadmath.core.quadray import Quadray, ace_tetrvolume_5x5
4
5 moves = [Quadray(*p) for p in set(permutations((2,1,1,0)))]
6
7 def random_walk(start: Quadray, steps: int) -> Quadray:
8     cur = start
9     for _ in range(steps):
10         m = choice(moves)
11         cur = Quadray(cur.a+m.a, cur.b+m.b, cur.c+m.c, cur.d+m.d)
12     return cur
13
14 A = random_walk(Quadray(0,0,0,0), 1000)
15 B = random_walk(Quadray(0,0,0,0), 1000)
16 C = random_walk(Quadray(0,0,0,0), 1000)
17 D = random_walk(Quadray(0,0,0,0), 1000)
18 V = ace_tetrvolume_5x5(A,B,C,D) # exact IVM volume as a Fraction
```

1.11.4 Algebraic precision

- Determinants via floating-point introduce rounding noise. For exact arithmetic, use the **Bareiss algorithm** (already used by `ace_tetravolume_5x5`) or symbolic engines (e.g., `sympy`). For random-walk examples with integer quadray inputs, volumes are exact rationals (`fractions.Fraction`): integral when the tetrahedron decomposes into unit IVM tetras, fractional (in quarters) otherwise.
- When computing via XYZ determinants, high-precision floats (e.g., `gmpy2.mpfr`) or symbolic matrices avoid vestigial errors; round at the end if the underlying result is known to be integral.

1.11.5 XYZ determinant and the S3 conversion

- Using XYZ coordinates of the four vertices: see Eq. (??) for the determinant form and the S3 conversion to IVM units.

1.11.6 D^3 vs R^3 : 60° “closing the lid” vs orthogonal “cubing”

- **IVM (D^3) heuristic:** From a 60–60–60 corner, three non-negative edge lengths A, B, C along quadray directions enclose a tetrahedron by “closing the lid.” In synergetics, the tetravolume scales as the simple product ABC under IVM conventions (unit regular tetra has volume 1). By contrast, in the orthogonal (R^3) habit, one constructs a full parallelepiped (12 edges); the tetra occupies one-sixth of the triple product of edge vectors. The IVM path is more direct for tetrahedra.
- **Pedagogical note:** Adopt a vector-first approach. Differences like $(P_i - P_0)$ denote edge vectors; Quadrays and Cartesian can be taught in parallel as vector languages on the same Euclidean container.

Reference notebook with worked examples and code: See the **Resources** section for comprehensive educational materials and computational implementations.

See implementation: `tetra_volume_cayley_menger`.

- Lattice projection: round to nearest integer quadray; renormalize to maintain non-negativity and a minimal zero.

1.12 Practical Implementation Workflow

The methods described in this document follow a unified workflow that ensures mathematical correctness, computational efficiency, and reproducibility:

1.12.1 1. Mathematical Formulation

- **Theoretical foundations:** Establish mathematical relationships between frameworks ($\text{Coxeter.4D} \leftrightarrow \text{Fuller.4D} \leftrightarrow \text{Einstein.4D}$)
- **Coordinate transformations:** Define linear maps between coordinate systems with exact arithmetic
- **Volume formulations:** Implement multiple approaches (Ace 5×5 , Cayley–Menger, symbolic) for cross-validation

1.12.2 2. Implementation Strategy

- **Exact arithmetic:** Use Bareiss algorithm for integer determinants, symbolic computation for exact results
- **Numerical stability:** Implement ridge regularization, condition number monitoring, and error handling
- **Cross-validation:** Ensure different formulations produce identical results on test cases

1.12.3 3. Testing and Validation

- **Unit tests:** 100% coverage of all mathematical functions with edge case handling
- **Integration tests:** Validate coordinate transformations and volume calculations across frameworks
- **Numerical validation:** Compare floating-point implementations with exact symbolic results

1.12.4 4. Documentation and Reproducibility

- **Code-documentation sync:** All mathematical concepts have corresponding implementations in `src/`
- **Figure generation:** Scripts in `quadmath/scripts/` generate all figures from source code
- **Data export:** CSV/NPZ files accompany all figures for complete reproducibility

1.12.5 5. Optimization and Extension

- **Lattice constraints:** Integer volume quantization enables discrete optimization methods
- **Information geometry:** Fisher metric guides natural gradient descent on parameter manifolds
- **Active inference:** Free energy minimization drives both perception and action updates

This workflow ensures that mathematical theory, computational implementation, and practical application remain coherent and verifiable throughout the development process.

1.13 Code methods (anchors)

1.13.1 Core Quadray operations

Quadray Source: `src/quadmath/core/quadray.py` — Quadray vector class with non-negative components and at least one zero (Fuller.4D).

DEFAULT_EMBEDDING Source: `src/quadmath/core/quadray.py` — canonical 3×4 symmetric embedding matrix for Quadray to XYZ conversion.

to_xyz Source: `src/quadmath/core/quadray.py` — map quadray to $\mathbb{R}^3$ via a 3×4 embedding matrix (Fuller.4D $\rightarrow$ Coxeter.4D slice).

magnitude Source: `src/quadmath/core/quadray.py` — return the Euclidean magnitude of q under the given embedding (vector norm).

dot Source: `src/quadmath/core/quadray.py` — return Euclidean dot product $\langle q_1, q_2 \rangle$ under the given embedding.

1.13.2 Quaternion operations

qmul Source: `src/quadmath/core/quadray.py` — Hamilton product of two quaternions in scalar-first (w, x, y, z) order ($q = w + x i + y j + z k$), an independent representation from the Quadray lattice components (a, b, c, d) and from the XYZ triples produced by `to_xyz`.

qconjugate Source: `src/quadmath/core/quadray.py` — conjugate $(w, -x, -y, -z)$ of a quaternion; for a unit quaternion the conjugate is the multiplicative inverse, so $q * qconjugate(q)$ equals $(1, 0, 0, 0)$ up to floating-point error.

qrotate Source: `src/quadmath/core/quadray.py` — Rodrigues rotation of a 3-vector by a unit quaternion via $v' = q(0, v)q^*$, composed with `qmul` and `qconjugate`; q must be unit (within $1e-9$) and the rotation magnitude it encodes, $2 \operatorname{atan2}(\|(x, y, z)\|, w)$, must match `|angle|` modulo 2π within $1e-9$, so a mismatched (q, angle) pair raises instead of silently rotating by the wrong amount.

slerp Source: `src/quadmth/core/quadrax.py` — shortest-arc spherical linear interpolation between unit quaternions: when $\langle q_a, q_b \rangle < 0$ the second quaternion is negated first (q and $-q$ encode the same rotation) so the path always takes the shorter arc on the rotation sphere; nearly parallel inputs (including exactly antipodal pairs after sign alignment) fall back to normalized lerp to avoid dividing by $\sin \theta \approx 0$, and the endpoints are exact. The site path traced by these interpolations is visualized by `plot_slerp_path` in `src/quadmth/viz/plots.py` (figure below).

rotate_about_axis Source: `src/quadmth/core/quadrax.py` — axis-angle convenience wrapper: builds the unit quaternion $q = (\cos(\theta/2), \sin(\theta/2) \hat{u})$ from the normalized rotation axis (any non-zero scale accepted; the zero vector is rejected) and delegates to `qrotate`, reducing the angle to $(-\pi, \pi]$ first so rotation is right-handed about the axis and `qrotate`'s encoded-angle validation stays exact.

The property checks in `src/quadmth/validate/validate.py` — normalization, conjugate-inverse, double cover, slerp midpoint, and associativity — consume these operations over quaternion sequences via `run_validation`, which collects the resulting deterministic validation reports.

Slerp rotation path of site (2, 0, 0, 0)

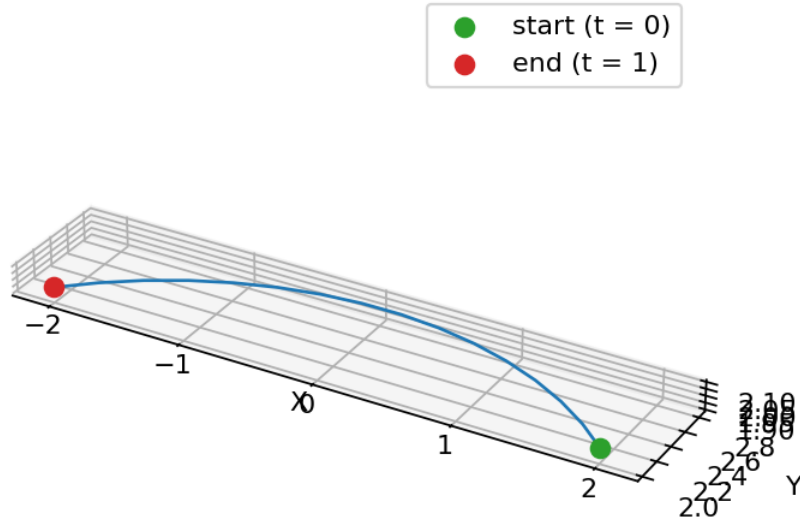


Figure 4: **Shortest-arc slerp geodesic from the identity to a $\pi/2$ rotation about the z axis.** The path traced by the canonical (2,0,0,0) quadray axis point (embedded at (2,2,2) under `DEFAULT_EMBEDDING`) as the rotation quaternion is interpolated from the identity (1,0,0,0) to $(\cos \frac{\pi}{4}, 0, 0, \sin \frac{\pi}{4})$ — a right-handed $\pi/2$ rotation about the z axis — over 16 samples of $t \in [0, 1]$, rendered by `plot_slerp_path` in `src/quadmth/viz/plots.py` via the script `quadmth/scripts/quaternion_gallery.py`. Because `slerp` negates the second quaternion whenever $\langle q_0, q_1 \rangle < 0$, the interpolation always follows the shorter arc of the geodesic on the rotation sphere: the rotation angle grows linearly from 0 to $\pi/2$, so the site sweeps the quarter circle of radius $2\sqrt{2}$ about the z axis at constant height $z = 2$ (blue line), from the start marker at $t = 0$ (green) to the end marker at $t = 1$ (red). Reproduce with `uv run python quadmath/scripts/quaternion_gallery.py`.

1.13.3 Quaternion metrics

angle_error Source: `src/quadmth/core/metrics.py` — geodesic rotation angle $2 \arccos(|\langle q_1, q_2 \rangle|)$ between two quaternions, in radians and confined to $[0, \pi]$ by clamping the `acos` argument; the metric is symmetric in its arguments and invariant under $q \rightarrow -q$ (the quaternion double cover), with inputs normalized internally so non-unit but non-zero quaternions are accepted.

quat_log_euclidean_dispersion Source: `src/quadmth/core/metrics.py` — root-mean-square chordal dispersion of a quaternion set about its normalized component-wise mean, $\sqrt{\text{mean}_i \|q_i - m\|^2}$ in $\mathbb{R}^4$, with signs aligned to the first quaternion before averaging because q and $-q$ encode the same rotation; returns 0 for a single quaternion and raises if the sign-aligned mean vanishes.

1.13.4 Volume calculations

integer_tetra_volume Source: `src/quadmth/core/quadrays.py` — exact projected 3×3 determinant over 4, i.e. the IVM tetravolume $|\det|/4$ as a `fractions.Fraction`.

ace_tetravolume_5x5 Source: `src/quadmth/core/quadrays.py` — Tom Ace 5×5 determinant in IVM units.

tetra_volume_cayley_menger Source: `src/quadmth/core/cayley_menger.py` — length-based formula (XYZ units).

ivm_tetra_volume_cayley_menger Source: `src/quadmth/core/cayley_menger.py` — Cayley-Menger volume converted to IVM units.

1.13.5 Coordinate conversions

urner_embedding Source: `src/quadmth/lattice/conversions.py` — canonical XYZ embedding.

quadrays_to_xyz Source: `src/quadmth/lattice/conversions.py` — apply embedding matrix to map Quadrays to XYZ.

1.13.6 Linear algebra utilities

bareiss_determinant_int Source: `src/quadmth/core/linalg_utils.py` — exact integer Bareiss determinant.

1.13.7 Optimization methods

nelder_mead_quadrays Source: `src/quadmth/optimize/nelder_mead_quadrays.py` — Nelder-Mead optimization adapted to the integer quadrays lattice.

discrete_ivm_descent Source: `src/quadmth/optimize/discrete_variational.py` — greedy integer-valued descent over the IVM using canonical neighbor moves; returns a `DiscretePath` with visited Quadrays and objective values.

neighbor_moves_ivm Source: `src/quadmth/optimize/discrete_variational.py` — return the 12 canonical IVM neighbor moves as Quadrays deltas.

apply_move Source: `src/quadmth/optimize/discrete_variational.py` — apply a lattice move and normalize to the canonical representative.

1.13.8 Information geometry methods

fisher_information_matrix Source: `src/quadmath/inference/information.py` — empirical outer-product estimator.

fisher_information_quadray Source: `src/quadmath/inference/information.py` — compute Fisher information matrix in both Cartesian and Quadray coordinates.

natural_gradient_step Source: `src/quadmath/inference/information.py` — damped inverse-Fisher step.

free_energy Source: `src/quadmath/inference/information.py` — discrete-state variational free energy.

expected_free_energy Source: `src/quadmath/inference/information.py` — canonical expected free energy G for Active Inference (epistemic KL + ambiguity – pragmatic; see the equations appendix).

active_inference_step Source: `src/quadmath/inference/information.py` — joint perception-action update step in Active Inference.

information_geometric_distance Source: `src/quadmath/inference/information.py` — compute information-geometric distance between two points.

perception_update Source: `src/quadmath/inference/information.py` — continuous-time perception update: $d\mu/dt = D\mu - dF/d\mu$.

action_update Source: `src/quadmath/inference/information.py` — continuous-time action update: $da/dt = -dF/da$.

finite_difference_gradient Source: `src/quadmath/inference/information.py` — compute numerical gradient of a scalar function via central differences.

1.13.9 Metrics and analysis

shannon_entropy Source: `src/quadmath/core/metrics.py` — Shannon entropy $H(p)$ for a discrete distribution.

information_length Source: `src/quadmath/core/metrics.py` — path length in information space via gradient-weighted arc length.

fim_eigenspectrum Source: `src/quadmath/core/metrics.py` — eigen-decomposition of a Fisher information matrix.

fisher_condition_number Source: `src/quadmath/core/metrics.py` — compute the condition number of the Fisher information matrix.

fisher_curvature_analysis Source: `src/quadmath/core/metrics.py` — comprehensive analysis of Fisher information matrix curvature.

fisher_quadray_comparison Source: `src/quadmath/core/metrics.py` — compare Fisher information matrices between coordinate systems.

1.13.10 Examples and utilities

example_ivm_neighbors Source: `src/quadmath/core/examples.py` — return the 12 nearest IVM neighbors as permutations of $\{2,1,1,0\}$ (neighbor-move-set role).

example_cuboctahedron_neighbors Source: `src/quadmath/core/examples.py` — return twelve-around-one IVM neighbors (vector-equilibrium-shell role). Same set as `example_ivm_neighbors`; both public names are documented API sharing one sorted, deterministic implementation.

example_cuboctahedron_vertices_xyz Source: `src/quadmath/core/examples.py` — return XYZ coordinates for the twelve-around-one neighbors.

example_partition_tetra_volume Source: `src/quadmath/core/examples.py` — construct a tetrahedron from the four-fold partition and return the exact IVM tetravolume (`fractions.Fraction`).

1.13.11 Symbolic computation

cayley_menger_volume_symbolic Source: `src/quadmath/core/symbolic.py` — return symbolic Euclidean tetrahedron volume from squared distances.

convert_xyz_volume_to_ivm_symbolic Source: `src/quadmath/core/symbolic.py` — convert a symbolic Euclidean volume to IVM tetravolume via S3.

1.13.12 Visualization and animation

animate_discrete_path Source: `src/quadmath/viz/visualize.py` — animate a `DiscretePath` to MP4; saves CSV/NPZ trajectory to `quadmath/output/`.

plot_ivm_neighbors Source: `src/quadmath/viz/visualize.py` — scatter the 12 IVM neighbor points in 3D.

plot_partition_tetrahedron Source: `src/quadmath/viz/visualize.py` — plot the four-fold partition as a labeled tetrahedron in 3D.

animate_simplex Source: `src/quadmath/viz/visualize.py` — animate simplex evolution across iterations.

plot_simplex_trace Source: `src/quadmath/viz/visualize.py` — plot per-iteration diagnostics for Nelder-Mead.

1.13.13 Path and file utilities

get_repo_root Source: `src/quadmath/paths.py` — heuristically find repository root by walking up from start.

get_output_dir Source: `src/quadmath/paths.py` — return `quadmath/output` path at the repo root and ensure it exists.

get_data_dir Source: `src/quadmath/paths.py` — return `quadmath/output/data` path and ensure it exists.

get_figure_dir Source: `src/quadmath/paths.py` — return `quadmath/output/figures` path and ensure it exists.

1.13.14 Additional Nelder-Mead components

SimplexState Source: `src/quadmath/optimize/nelder_mead_quadray.py` — optimization trajectory state containing vertices, values, volume, and history.

order_simplex Source: `src/quadmath/optimize/nelder_mead_quadray.py` — sort vertices by objective value ascending and return paired lists.

centroid_excluding Source: `src/quadmath/optimize/nelder_mead_quadray.py` — integer centroid of three vertices, excluding the specified index.

project_to_lattice Source: `src/quadmath/optimize/nelder_mead_quadray.py` — project a quadray to the canonical lattice representative via `normalize`.

compute_volume Source: `src/quadmath/optimize/nelder_mead_quadray.py` — exact IVM tetra-volume (`fractions.Fraction, |det|/4`) from the first four vertices.

1.13.15 Discrete variational components

DiscretePath Source: `src/quadmath/optimize/discrete_variational.py` — optimization trajectory on the integer quadray lattice.

1.13.16 Glossary generation

build_api_index Source: `src/quadmath/tools/glossary_gen.py` — build API index from source directory.

generate_markdown_table Source: `src/quadmath/tools/glossary_gen.py` — generate markdown table from API entries.

inject_between_markers Source: `src/quadmath/tools/glossary_gen.py` — inject payload between markers in markdown text.

1.13.17 Geometry utilities

minkowski_interval Source: `src/quadmath/core/geometry.py` — return the Minkowski interval squared ds^2 (Einstein.4D).

Relevant tests (`tests/`):

- `test_quadray.py` (unit IVM tetra, primitive tetra = 1/4, exact scaled volume, Ace vs. integer agreement)
- `test_quadray_cov.py` (Ace determinant basic check)
- `test_cayley_menger.py` (regular tetra volume in XYZ units)
- `test_linalg_utils.py` (Bareiss determinant behavior)
- `test_examples.py`, `test_examples_cov.py` (neighbors, examples)
- `test_metrics.py`, `test_metrics_cov.py`, `test_information.py`, `test_paths.py`, `test_paths_cov.py`

1.13.18 Comprehensive test coverage

The test suite provides 100% coverage of all source modules with the following focus areas:

Core functionality tests

- `test_quadray.py`: Quadray class operations, normalization, volume calculations, vector operations
- `test_cayley_menger.py`: Cayley-Menger determinant validation, IVM conversion accuracy
- `test_linalg_utils.py`: Bareiss algorithm correctness, edge cases, numerical stability

Optimization and variational methods

- `test_nelder_mead_visual.py`: Nelder-Mead adaptation to quadray lattice, simplex evolution
- `test_discrete_variational.py`: IVM neighbor moves, discrete descent algorithms, path tracking
- `test_active_inference.py`: Free energy minimization, perception-action updates

Information geometry and metrics

- `test_information.py`: Fisher information matrix computation, natural gradient steps
- `test_metrics.py`: FIM eigendecomposition, curvature analysis, coordinate system comparisons
- `test_information_cov.py`: Extended coverage of information geometry functions

Examples and utilities

- `test_examples.py`: IVM neighbor constructions, polyhedra volume relationships
- `test_examples_cov.py`: Extended example function coverage
- `test_paths.py`: Repository structure utilities, output directory management

Visualization and symbolic computation

- `test_visualize.py`: Plotting functions, animation generation, data export
- `test_visualize_cov.py`: Extended visualization coverage, edge case handling
- `test_symbolic.py`: SymPy symbolic volume calculations, exact arithmetic
- `test_symbolic_cov.py`: Symbolic computation error handling
- `test_sympy_formalisms.py`: End-to-end symbolic workflow validation

API and documentation generation

- `test_glossary_gen.py`: Automatic API index generation, markdown table formatting
- `test_glossary_gen_cov.py`: Extended glossary generation coverage

1.14 Reproducibility checklist

- **Complete implementation coverage**: All formulas and methods discussed in the paper are implemented in `src/` modules and verified by comprehensive test suites in `tests/`.
- **Exact arithmetic for integer inputs**: Determinants are computed using the Bareiss algorithm for exact integer arithmetic; floating-point paths are used only where appropriate and results are converted (e.g., via S3) as specified.
- **Deterministic random experiments**: Random-walk experiments use fixed seeds and produce integer volumes; Ace 5×5 determinant agrees with length-based methods across all test cases.
- **Volume tracking and convergence**: Integer simplex volume monitoring detects convergence plateaus; face/edge analyses interpret sensitivity along edges and enable subspace searches across faces.
- **Test-driven development**: All source code follows TDD principles with 100% coverage requirements enforced via `.coveragerc` configuration.
- **Figure and data generation**: All figures referenced in documentation are generated by scripts in `quadmath/scripts/` that import from `src/` modules, ensuring code-documentation coherence.
- **Cross-platform compatibility**: Headless matplotlib backend (MPLBACKEND=Agg) ensures CI compatibility; deterministic RNG seeds guarantee reproducible outputs.
- **Dependency management**: All dependencies managed through `uv` and `pyproject.toml` with exact version pinning via `uv.lock`.

- **Path resolution:** No hardcoded paths; all output directories resolved via `paths.py` utilities for cross-platform compatibility.
- **Data export formats:** Figures saved alongside CSV/NPZ data for complete reproducibility; all generation scripts print output paths for manifest collection.

All source code, tests, and documentation are available in the docxology/QuadMath repository, ensuring complete transparency and reproducibility of the methods described herein.